Tutorial desempakado SVKP V1.3x

Programa:	**Download Accelerator Plus v7.2**
Protección:	SVKP v1.3 + Trial 30 días + nagscreen
Objetivos:	Desempakar SVKP v1.3x
URL descarga:	http://www.softpointer.com/tr.htm
Dificultad:	Tiene su miga
Herramientas:	OllyDbg 1.10 + PEiD 0.94 + ImpREC 1.6 + MSWord + HexWorkShop 4.23 PEditor 1.7 + Calculadora cybult 1.5
Cracker:	SyXe'05 .::[CracksLatinoS]::. Fecha: 09-09-06

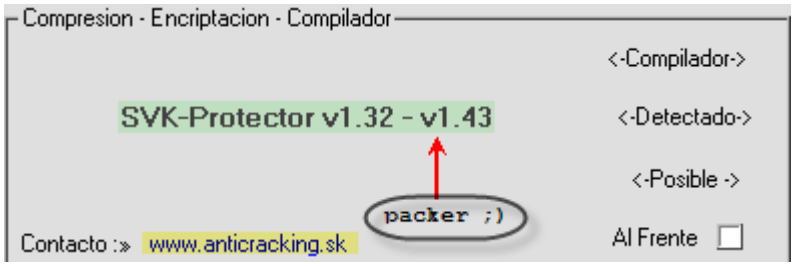
Hola amigos de CracksLatinoS!!

El tute de hoy trata sobre el desempaquetado o desempakado queramos. El caso es que hace un tiempo atrás estuvimos Solid (al que le mando un abrazo XDD) y un servidor mirando unos programitas Trial que bajó de Internet para entretenernos y topamos con uno que estaba empaquetado, en un principio intentamos desempakarlo pero como éste no era el objetivo seguimos con otro, el caso es que aún así yo lo intenté desempakar pero no pude¡!!! Se lo dije a Solid y nos fuimos a por otro. Esa es la historia de este programa. Ahora bien, como no lo borré de mi disco duro hace unos días lo encontré por casualidad y me dije ¿por qué no mirarlo con más calma? Y eso es lo que he estado haciendo últimamente, os diré que dentro de los SVKP que encontré este fue de largo el que más me ha costado, ya veremos porqué XDDD

AL atake…

Hecha la oportuna introducción vamos al "cracking" lo primero como siempre es pasarle un identificador tipo PEiD o RDG packer, lo veremos con ambos:

RDG packer v0.5.7:

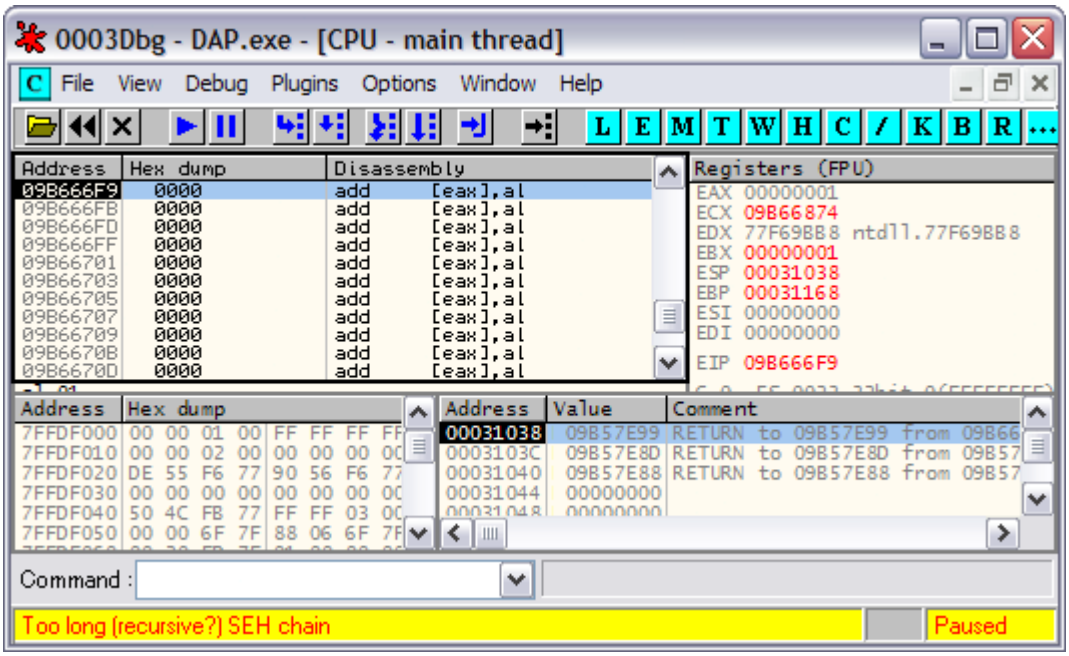


PEiD v0.92 [El v0.94 no lo encuentro ;( ]

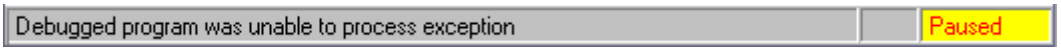
Lo dicho, ambos detectores apuntan lo mismo, se trata sin duda de SVKP v1.3x a partir de ahora comienza el estudio, los que me conocen ya saben que me gusta entrar bastante a fondo en el packer y entender en lo máximo posible su funcionamiento. Visto el enemigo vamos a por él ;)

Preparando al Olly…

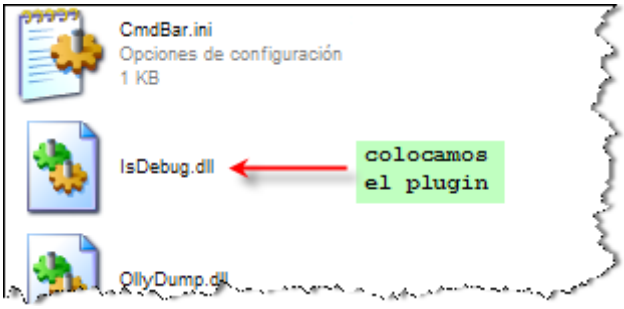
En este punto lo que haremos será configurar el Olly adecuadamente y agregarle los plugins necesarios para que el programa corra sin problemas, pues si probamos a ejecutarlo así sin más ocurre esto:



Nos dice que el SEH es demasiado grande y entre paréntesis dice ¿recursivo? Sabemos que una función recursiva es aquella que se invoca a sí misma, la barra de estado parpadea y cambia entre este mensaje y otro tipo STACK OVERFLOW. Además se ve la ventana de código toda llena de ceros ¿? Si pasamos la excepción con SHIFT+F9 las cosas empeoran, y en la de estado veremos otro mensaje de este estilo:



Acompañado del "Too long SHE…" que veíamos antes y Olly queda aquí estancado, por suerte esto tiene fácil solución, se trata del byte de detección del debugger que se lee con la api IsDebuggerPresent, tenemos la opción de cambiar dicho byte manualmente o bien dotar a Olly del plugin necesario para superar la detección, como ya está más que explicado como ocultarlo a mano le colocamos el plugin y listos, no debemos preocuparnos de nada más el programa ya corre perfecto, le colocamos el plugin:



Ahora mismo yo tengo estos puestos:

1.- CmdBar.dll
2.- IsDebug.dll
3.- OllyDump.dll
4.- OllyScript.dll

La verdad es que no soy partidario de tener demasiados, sobre todo si no los necesito, si por ejemplo me hiciera falta el MapConv (MapConverter) pues reinicio Olly y se lo pongo, nada más.

Aclarado esto seguimos, cargamos el EXE protegido en Olly y para en su EP:

```
Address   Hex dump        Disassembly                          Comment
005EE000  60              pushad          <------ EntryPoint
005EE001  E8 00000000     call    005EE006
005EE006  5D              pop     ebp
005EE007  81ED 06000000   sub     ebp,6
005EE00D  EB 05           jmp     short 005EE014
005EE00F  B8 72C44300     mov     eax,0043C472
005EE014  64:A0 23000000  mov     al,byte ptr fs:[23]
005EE01A  EB 03           jmp     short 005EE01F
005EE01C  C784E8 84C0EB0  mov     dword ptr ds:[eax+ebp*8+3EB
005EE027  67:B9 49000000  mov     ecx,49                       Superfl
005EE02D  8DB5 C5020000   lea     esi,dword ptr ss:[ebp+2C5]
005EE033  56              push    esi
005EE034  8006 44         add     byte ptr ds:[esi],44
005EE037  46              inc     esi
005EE038  E2 FA           loopd   short 005EE034
005EE03A  8B8D C1020000   mov     ecx,dword ptr ss:[ebp+2C1]
005EE040  5E              pop     esi
005EE041  55              push    ebp
005EE042  51              push    ecx
```

Desmarcamos todas las casillas en OPTIONS - DEBUGGING OPTIONS - EXCEPTIONS para que Olly se vaya deteniendo en ellas, y dejamos marcada esta:

☑ Ignore memory access violations in KERNEL32

Pulsamos F9 y vamos pasando las excepciones con SHIFT+F9 hasta que pasada una cargue el programa, anotamos esa que será la última, este método es de Ricardo Narvaja [muy bueno por cierto], la última en mi máquina es ésta:

```
Address   Hex dump        Disassembly                          Comment
01566ED1  8DC0            lea     eax,eax                      Illegal use of register
01566ED3  EB 02           jmp     short 01566ED7
01566ED5  0F8D 9DEB03C7   jge     C85A5A78
01566EDB  84E8            test    al,ch
01566EDD  0F85 A0FEFFFF   jnz     01566D83
01566EE3  E8 01000000     call    01566EE9
01566EE8  9A 83C40464 8F  call    far 058F:6404C483            Far call
01566EEF  0000            add     [eax],al
01566EF1  0000            add     [eax],al
01566EF3  5B              pop     ebx
01566EF4  E8 02000000     call    01566EFB
01566EF9  CD 20           int     20
```

Si ahora ponemos un BPM en la sección de código y damos Run para aquí:

```
Address   Hex dump        Disassembly                          Comment
004A9ECA  6A 00           push    0
004A9ECC  FF15 8CB55100   call    dword ptr ds:[51B58C]
004A9ED2  E8 67100500     call    004FAF3E                     jmp to MFC42.#6438
004A9ED7  FF15 C4B25100   call    dword ptr ds:[51B2C4]
004A9EDD  8985 6CFFFFFF   mov     dword ptr ss:[ebp-94],eax
004A9EE3  8D4D 90         lea     ecx,dword ptr ss:[ebp-70]
004A9EE6  E8 CB430000     call    004AE2B6
004A9EEB  C645 FC 0F      mov     byte ptr ss:[ebp-4],0F
004A9EEF  8D45 90         lea     eax,dword ptr ss:[ebp-70]
004A9EF2  50              push    eax
004A9EF3  8B8D 70E9FFFF   mov     ecx,dword ptr ss:[ebp-1690]
004A9EF9  E8 3A100500     call    004FAF38                     jmp to MFC42.#5214
004A9EFE  837D BC 00      cmp     dword ptr ss:[ebp-44],0
004A9F02  74 69           je      short 004A9F6D
004A9F04  B9 54605700     mov     ecx,00576054
004A9F09  E8 3DD0F8FF     call    00436F4B
004A9F0E  8B8D 70E9FFFF   mov     ecx,dword ptr ss:[ebp-1690]
004A9F14  E8 86250000     call    004AC49F
004A9F19  83A5 78EAFFFF   and     dword ptr ss:[ebp-1588],0
004A9F20  C645 FC 0E      mov     byte ptr ss:[ebp-4],0E
004A9F24  8D4D 90         lea     ecx,dword ptr ss:[ebp-70]
004A9F27  E8 2E440000     call    004AE35A
004A9F2C  C645 FC 0D      mov     byte ptr ss:[ebp-4],0D
004A9F30  8D8D 74FFFFFF   lea     ecx,dword ptr ss:[ebp-8C]
004A9F36  E8 [illegible]  call    004FA056                     jmp to MFC42.#6307
```

He aquí el primer dolor de cabeza, aparentemente estamos en el OEP, vemos el típico PUSH 0 de los ejecutables compilados en ASM, si miramos la pila vemos que está totalmente descuadrada como si hubieran stolen bytes, además sabemos que este packer suele agregar stolen y aquí no los vemos:

Veamos la pila [stack]:

```
0012E7FC  005791E0 DAP.005791E0
0012E800  005791E0 DAP.005791E0
0012E804  FFFFFFFF
0012E808  77F53720 RETURN to ntdll.77F53720 from ntdll.77F4183D
0012E80C  00150000
0012E810  77F5379E RETURN to ntdll.77F5379E from ntdll.77F68C4E
0012E814  001542B0
0012E818  00150000
0012E81C  0015FE00
0012E820  77F42013 RETURN to ntdll.77F42013
0012E824  77F4201C RETURN to ntdll.77F4201C from ntdll.77F68C4E
0012E828  0012EB6C
0012E82C  0000001C
0012E830  0000020D
0012E834  0015FBA8
0012E838  00150178
0012E83C  77F42013 RETURN to ntdll.77F42013
```

En realidad el puntero de pila [ESP] debería estar en otra posición, sabemos que el S.O. carga el programa como si fuera un hilo o "thread" y desde su propio código hace un call que carga el programa, éste no varía nunca de posición por lo que la dirección de retorno será siempre la misma en cada máquina, aunque en otra distinta ésta puede variar, lo que debemos hacer es buscar ese valor en la pila para ver donde comenzó la auténtica ejecución del programa, miremos al stack otra vez:

```
0012FFB8  00530CF0 DAP.00530CF0
0012FFBC  00000000
0012FFC0  0012FFF0
0012FFC4  77E614C7 RETURN to kernel32.77E614C7
0012FFC8  004B1951 DAP.004B1951
0012FFCC  0012CD54
0012FFD0  7FFDF000
0012FFD4  F6E98CF0
0012FFD8  0012FFC8
0012FFDC  8053D14F
0012FFE0  FFFFFFFF End of SEH chain
0012FFE4  77E74809 SE handler
0012FFE8  77E71210 kernel32.77E71210
0012FFEC  00000000
0012FFF0  00000000
0012FFF4  00000000
0012FFF8  005EE00A offset DAP.<ModuleEntryPoint>
```

Ahí vemos la dirección citada, en mi caso se trata de :77E614C7 y no varía en ningún ejecutable, cada vez que cargo uno en Olly es el mismo, lo que hay de ahí para arriba serían los supuestos stolen bytes y comenzarían a partir de :0012FFC0 pero nosotros vemos un PUSH 0 claramente y no los stolen así que ese no es nuestro OEP ;(

Lo que haremos ahora será poner el BPM on memory Access en la sección de código una excepción antes para localizarlo, reiniciamos Olly [CTRL+F2] y llegamos a esta excepción:

```
Address   Hex dump     Disassembly          Comment
013ADC09  CD 01        int     1
013ADC0B  E8 01000000  call    013ADC11
013ADC10  E9 83C4047C  jmp     7D3FA098
013ADC15  03EB         add     ebp,ebx
013ADC17  039A 74FB648F add    ebx,[edx+8F64FB74]
013ADC1D  05 00000000  add     eax,0
013ADC22  E8 02000000  call    013ADC29
013ADC27  CD 20        int     20
013ADC29  830424 08    add     dword ptr [esp],8
```

Ponemos el BPM en la sección de código [la siguiente al PE header], supongo que hacer esto no será ningún problema, la superamos con SHIFT+F9 y Olly se detiene por primera vez aquí:

```
Address   Hex dump     Disassembly          Comment
013CF14F  8A06         mov     al,[esi]     ; esta leyendo..
013CF151  46           inc     esi
013CF152  47           inc     edi
013CF153  8843 0F      mov     [ebx+F],al
013CF156  8A46 FF      mov     al,[esi-1]
013CF159  55           push    ebp
013CF15A  E8 00000000  call    013CF15F
013CF15F  5D           pop     ebp
013CF160  81ED 0D470000 sub    ebp,470D
```

Paró por lectura [when reading..] y además está en un loop donde va leyendo de la sección de código que comienza en :401000 guarda el byte tomado en una sección del tipo :013A0000 en mi máquina, carga una llave de desencriptación en CL con la que desencripta dicho byte en este loop:

```
013CF166    8A8D 50030000   mov     cl,[ebp+350]          // carga llave encriptación
013CF16C    5D              pop     ebp                   // desencripta el byte
013CF16D    32C1            xor     al,cl                 // salva en sección .code
013CF16F    8847 FF         mov     [edi-1],al
013CF172    8BC5            mov     eax,ebp
013CF174    4D              dec     ebp
013CF175    85C0            test    eax,eax
013CF177  ^ 75 A4           jnz     short 013CF11D        // al inicio del loop..
```

Lo que hace es desencriptar código, quitamos el BPM que tenemos puesto y colocamos un Bp al final del bucle, pulsamos F2 y lo superamos, volvemos a poner el BPM en la sección de código y damos RUN (F9) para caer aquí:

```
Address   Hex dump        Disassembly                          Comment
013C99AE    66:813E 4A43  cmp     word ptr [esi],434A          ; busca "JC"
013C99B3  v 0F85 23010000 jnz     013C9ADC
013C99B9    83C6 0A       add     esi,0A
013C99BC    C745 FC 080000( mov   dword ptr [ebp-4],8
013C99C3    33DB          xor     ebx,ebx
013C99C5    BA 00000080   mov     edx,80000000
013C99CA    43            inc     ebx
013C99CB    33C0          xor     eax,eax
013C99CD    E8 65010000   call    013C9B37
```

Compara el puntero de ESI con 0x434Ah que en ASCII corresponde a "JC" como podemos ver en la ventanita de dump [zona de datos]:

```
Address   Hex dump                                         ASCII
00401000  4A 43 00 90 11 00 F5 3F 1C A6 7E 71 D5 20 A0 02  JC.É◄.§?└ª~q'
00401010  9F C9 1C EB FE 87 11 C2 C8 00 43 08 22 44 B5 8D  ƒ╔└Lù■ç◄┬╚.C■"D
00401020  1D 2D FB A3 8F 44 AE 91 87 8E 77 84 A3 F0 1C E8  #-¹ûÃD«æçÄwäú
00401030  0D 15 3A D6 CB 70 DD 50 F0 D4 DF C1 50 E8 B6 DA  .§:íÓp¦P-É═┴Pþ
00401040  10 61 AA 75 E3 2F 48 D2 F1 FF DC 38 12 09 D1 62  ►a¬uõ/HÉ± ▄8$.
00401050  0D 30 FC 0B BC B0 FF 0D D5 E2 22 05 A4 DD 6A A0  .0ª♂ª▒ .'Ó"♣ñ¡j
00401060  0D B3 AA 46 07 09 47 D8 14 10 B8 25 40 A3 CB 0E  .|¬F•.Gï¶►Ø%@ú╦
00401070  36 B8 89 AA 03 FC 21 FF 77 BE 24 41 0B D8 0F 34  6@ë¬♥ª ! w¥$Aσï☼
00401080  FC 3B 70 BD C4 5A 16 35 58 E0 FB 1D 50 05 E9 37  ª;p¢-Z▬5XÓ¹#P♣
00401090  D5 05 61 62 [illegible] 5E 93 47 [illegible] 21 34 [illegible] 82 [illegible] C2 FA [illegible]  '♣ab[illegible]
```

Si somos curiosos nos daremos cuenta que esta zona es la recién encriptada por lo que deduzco que está verificando si todo fue ido bien en la desencriptación como este es el caso seguimos en la rutina en la que hay muchos call, si vamos pulsando F9 iremos parando cada vez en este instrucción:

```
Address   Hex dump   Disassembly                Comment
013C9B3B    8B16     mov     edx,[esi]
013C9B3D    83C6 04  add     esi,4
013C9B40    F9       stc
013C9B41    13D2     adc     edx,edx
013C9B43    C3       ret
```

Estamos en otro loop, vamos viendo cómo ESI comienza con valor :401000 y se va incrementando de a 1, si pasamos el RET llevamos a otro bucle que realiza con la instrucción LOOPD y en el siguiente RET nos pone aquí:

```
Address   Hex dump   Disassembly                Comment
013C99DC    0245 F7  add     al,[ebp-9]
013C99DF    AA       stosb                     ; escribe los bytes
013C99E0  ^ EB E9    jmp     short 013C99CB
013C99E2    E8 50010000 call  013C9B37
013C99E7  v 0F82 97000000 jb  013C9A84
```

En ese "STOS BYTE PTR ES:[EDI]" lo que hace es copiar el byte contenido en AL allí donde apunte EDI [o sea, en *EDI], que apunta a una zona temporal solicitada en tiempo de ejecución por el packer tipo :01410000 en mi máquina como podemos ver en la siguiente imagen:

```
Address   Hex dump    Disassembly                    Comment
01410000  55          push   ebp
01410001  8BEC        mov    ebp,esp
01410003  33C0        xor    eax,eax
01410005  50          push   eax
01410006  50          push   eax
01410007  50          push   eax
01410008  FF75 1C     push   dword ptr [ebp+1C]
0141000B  FF75 18     push   dword ptr [ebp+18]
0141000E  FF75 14     push   dword ptr [ebp+14]
01410011  FF75 10     push   dword ptr [ebp+10]
```

Lo que está haciendo es escribir código desencriptado en esta sección temporal y si miramos para abajo encontramos el final del bucle:

```
Address   Hex dump    Disassembly                    Comment
013C9ACF  56          push   esi
013C9AD0  8BF7        mov    esi,edi
013C9AD2  2BF0        sub    esi,eax
013C9AD4  F3:A4       rep    movsb                   ; copia bytes
013C9AD6  5E          pop    esi
013C9AD7  E9 EFFEFFFF jmp    <inicio_loop>
013C9ADC  8B75 08     mov    esi,[ebp+8]             ; BP aqui
013C9ADF  837E 06 00  cmp    dword ptr [esi+6],0
013C9AE3  74 46       je     short 013C9B2B
013C9AE5  56          push   esi
```

Cuando rompe en el Bp ya tenemos toda la zona desencriptada, esos bytes irán después en la sección de código del ejecutable protegido, si echamos un vistazo incluso podemos ver los saltos a la iat:

```
Address   Hex dump    Disassembly                    Comment
01509A41  25 94B05100 and    eax,51B094
01509A46  FF25 14A95100 jmp  [51A914]
01509A4C  FF25 18A95100 jmp  [51A918]
01509A52  FF25 1CA95100 jmp  [51A91C]
01509A58  FF25 20A95100 jmp  [51A920]
01509A5E  FF25 24A95100 jmp  [51A924]
01509A64  FF25 28A95100 jmp  [51A928]
01509A6A  FF25 2CA95100 jmp  [51A92C]
01509A70  FF25 30A95100 jmp  [51A930]
01509A76  FF25 34A95100 jmp  [51A934]
```

Se ve claramente que estas instrucciones irán posteriormente en nuestro .exe, lógicamente en la sección de código como ya he dicho antes, si ahora ponemos de nuevo el BPM en dicha sección y apretamos F9 iremos parando en instrucciones donde copia dicho código en ella:

```
Address   Hex dump    Disassembly                    Comment
013C9B75  83E2 03     and    edx,3
013C9B78  C1E9 02     shr    ecx,2
013C9B7B  F3:A5       rep    movsd                   ; copia codigo..
013C9B7D  03CA        add    ecx,edx
013C9B7F  F3:A4       rep    movsb
013C9B81  5A          pop    edx
013C9B82  81C2 00040000 add  edx,400
013C9B88  8BC8        mov    ecx,eax
013C9B8A  EB 02       jmp    short 013C9B8E
```

Aquí es donde copia la mayor parte, si pasamos esta zona nos topamos de bruces con un pequeño e inofensivo enemigo: los RDTSC..

```
Address   Hex dump    Disassembly                    Comment
013C9B98  33C0        xor    eax,eax                 DAP.0055CFE4
013C9B9A  EB 02       jmp    short 013C9B9E
013C9B9C  CD 20       int    20
013C9B9E  E8 8D000000 call   013C9C30
013C9BA3  C3          ret
013C9BA4  EB 84       jmp    short 013C9BAA
```

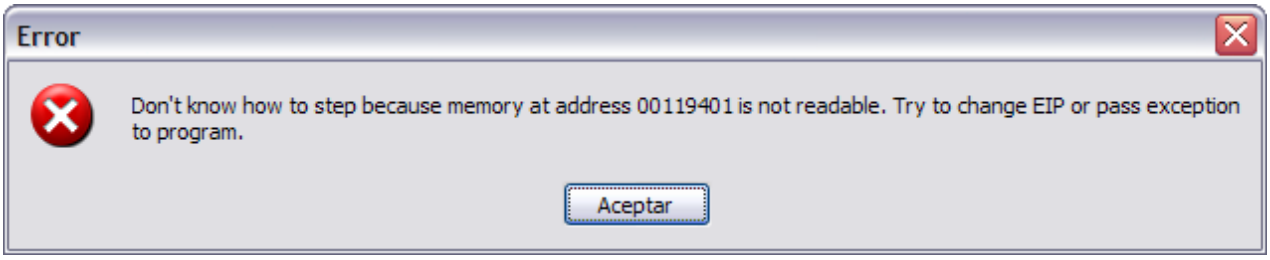
Lo supero con F8 y veo esto en EAX:

```
Registers (FPU)
EAX F2893F0C  <-- tiene el TimeDateStamp
ECX 09A00000
EDX 00003A8F
EBX 099B9B6D
ESP 0012E400
EBP 09990000
```

Como pongo en el comentario de la imagen ese valor es el ***TimeDateStamp***, no sé bien cómo definirlo pero sí puedo asegurar que en este caso es utilizado como técnica antidebug, por lo visto va tomando tiempos, algo similar a lo que hace GetTickCount pero de otra manera dejando el resultado en EAX, de momento seguimos traceando ignorando dicha instrucción y llegamos a:

```
Address   Hex dump   Disassembly          Comment
013C9C39  EB 02      jmp   short 013C9C3D
013C9C3B  CD 20      int   20
013C9C3D  C3         ret                  ; este RET nos mata ;(
013C9C3E  55         push
```

Si pasamos el RET con F8 la pantalla de Olly se queda en blanco diciendo:



Parece que en un momento concreto el EIP toma un valor erróneo e intenta saltar a posiciones de memoria que no están preparadas y por su puesto falla generando un #***PF*** [Page Fault] conocida como ***EXCEPTION_ACCESS_VIOLATION***, o sea una violación de acceso a memoria normal y corriente, la solución pasa por modificar los rdtsc, tarea bien sencilla de no ser por esto:

```
Address   Disassembly          Comment
013A0000  add   [eax],al       (Initial CPU selectic
013A1CB0  rdtsc
013A1D05  rdtsc
013A3276  rdtsc
013A32CB  rdtsc
013A35DC  rdtsc
013A3631  rdtsc
013A3702  rdtsc
013A3757  rdtsc
013A5C48  rdtsc
013A5C9D  rdtsc
013A5DAC  rdtsc
013A5E01  rdtsc
013A5ED8  rdtsc
013A5F2D  rdtsc
013A5FD2  rdtsc
013A6027  rdtsc
013A612F  rdtsc
013A6184  rdtsc
013A625C  rdtsc
013A62B1  rdtsc
013A6354  rdtsc
013A63A9  rdtsc
013A64BC  rdtsc
013A6511  rdtsc
013A65FB  rdtsc
```

Listado de rdtsc

Esta lista nos será de utilidad después para evitar esta técnica anti-traceo...

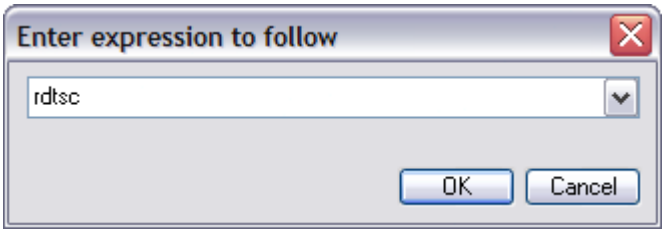
Está todo el código repleto de éstos, la lista es inmensa, tracear esta zona puede resultar de lo más complicado, podríamos con un pequeño injerto reemplazarlas con instrucciones menos dañinas, como por ejemplo "mov al, 1" o "xor eax, eax" que ocupan dos bytes cada una, optaremos por la segunda:

- Cambiaremos todos los "rdtsc" por "xor eax,eax"

Reiniciamos Olly y llegamos nuevamente al primero de ellos:

```
Address   Hex dump   Disassembly          Comment
013C9B98  0F31       rdtsc                ; primer rdtsc
013C9B9A  EB 02      jmp   short 013C9B9E
013C9B9C  CD 20      int   20
013C9B9E  E8 8D000000 call 013C9C30
013C9BA3  C3         ret
013C9BA4  EB 04      jmp   short 013C9BAA
```

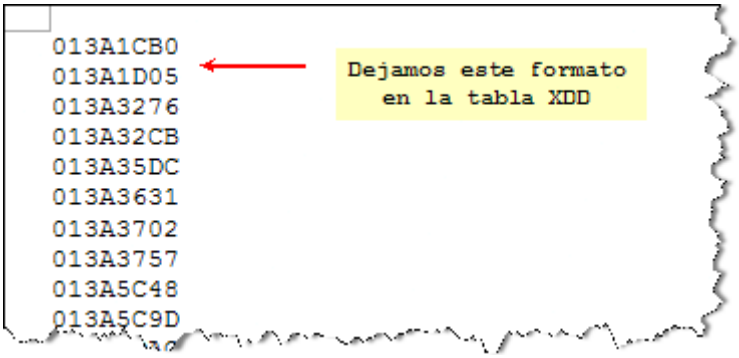
Ahora nos desplazamos al inicio de esta sección con "Ctrl+Inicio" y desde allí hacemos una búsqueda de todos ellos con SEARCH FOR - ALL COMMANDS y ponemos:



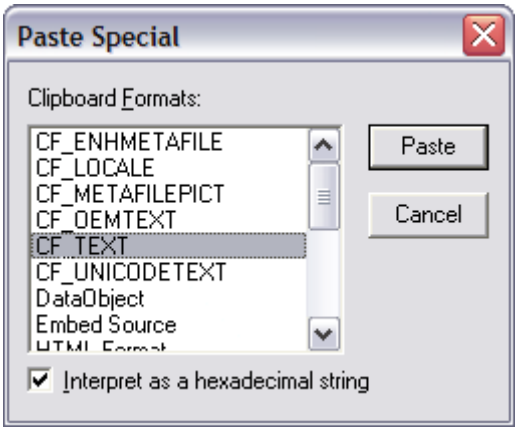
Pulsamos en "***Find***" y en la ventana de "***References***" obtendremos un listado con la dirección de todos ellos, esto lo hacemos para evitar parchear en una posición errónea donde se encuentren los mismos bytes ***0x0F31h***, obtenemos una lista que ya vimos antes y que después deberemos modificar para utilizarla en nuestro injerto, lo haremos de esta manera:

1.- Copiamos la tabla completa al portapapeles "***Copy to clipboard->Whole table***"
2.- Abrimos en ***Word*** y la pegamos completa
3.- Reemplazamos la cadena " ***rdtsc***" por espacios en blanco
4.- Eliminamos las líneas iniciales que no son direcciones válidas

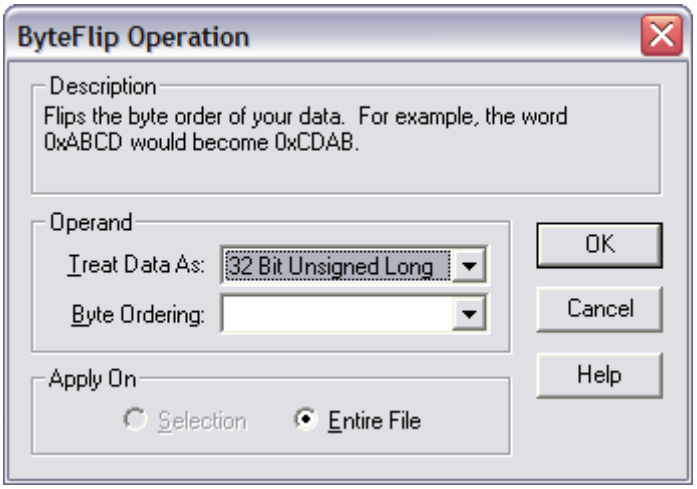
Hecho esto tendremos una tabla con el siguiente formato:



Abrimos el Hex WorkShop v4.23 (el que tengo yo), seleccionamos la tabla completa, la copiamos con "***Ctrl+C***", nos vamos al HWShop, creamos un nuevo archivo con "***Ctrl+N***" y elegimos "***Edit->Paste Special***" así:



Esta es la configuración que dejaremos, pulsamos PASTE y se insertarán todos los bytes, pero falta algo para poderlos utilizar en el Olly, seleccionamos la opción TOOLS - OPERATIONS - BYTE FLIP y elegimos 32 BIT UNSIGNED LONG como se ve en la siguiente imagen:



Ahora apretamos en "OK" y ya lo tenemos preparado, podemos salvar el archivo como "rdtsc.hex" por ejemplo.

Cuando ya tenemos el archivo preparado retornamos a Olly, estábamos justo en el primer rdtsc de :013C9B98 en mi máquina, debemos buscar un hueco al injerto, pero antes buscamos un lugar para albergar la tabla, encontramos uno a partir de :0055C890 y para colocar el injerto encontramos un espacio libre y suficiente a partir de :013EB200 quedando algo así:

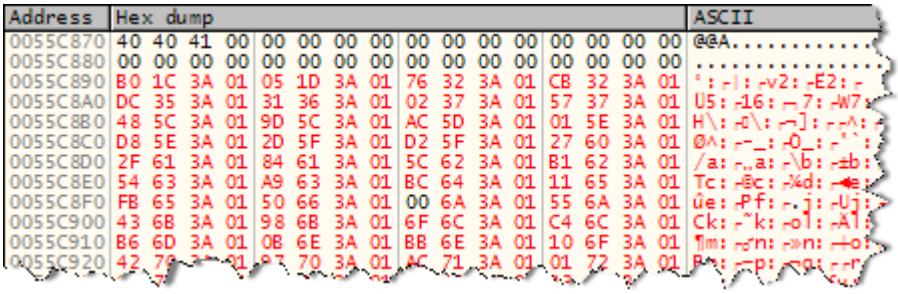
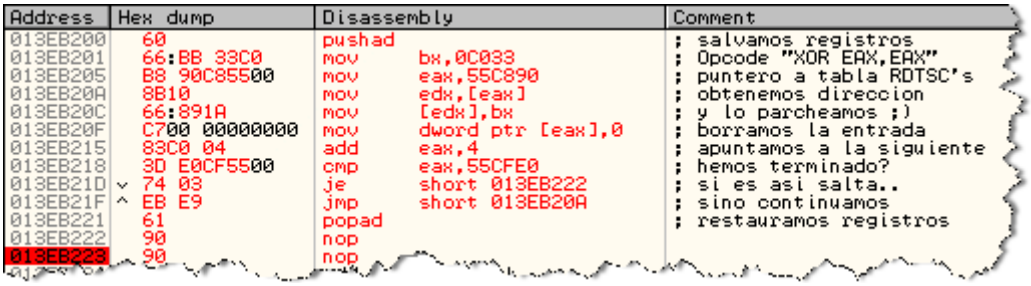


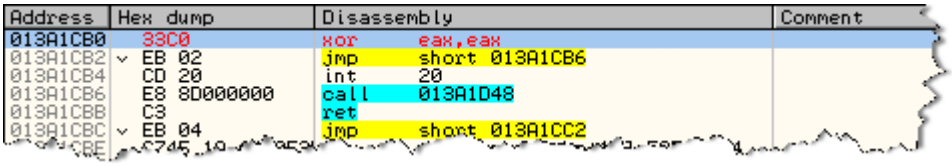
Tabla de rdtsc

Esta tabla contiene la dirección de todos los rdtsc que hay en el código de la nueva sección temporal que el packer solicita al ejecutarse

Lo único que nos falta es injertar, quedaría algo así:



Es pequeño pero efectivo, situamos el EIP en la primera instrucción del mismo con NEW ORIGIN HERE y lo lanzamos sin olvidar el BP después del POPAD para parar allí una vez termine, pulsamos F9 y comenzará el trabajo, en breve para miramos la tabla vemos que hizo bien su trabajo pero quedó un rdtsc por resolver en :013EAE42 que modificamos a mano, para que no vuelva a suceder modificamos la línea : 013EB218 del injerto cambiando el 55CFE0 por 55CFE4 y problema resuelto, echando un vistazo a las direcciones donde había algún rdtsc como esta vemos que están reemplazados:



Vemos que fue todo bien, miremos también cómo quedó la tabla:

```
Address   Hex dump                                          ASCII
0055C890  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
0055C8A0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
0055C8B0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
0055C8C0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
0055C8D0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
0055C8E0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
0055C8F0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
0055C900  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
0055C910  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
```

totalmente vacía [excepto el valor de :55CFE0 borrado a mano], lo que indica que no queda ningún rdtsc en el código, quitamos el Bp del injerto, lo borramos con UNDO SELECTION y restauramos el EIP adonde de estaba antes -> :013C9B98 dejándolo allí de nuevo:

```
Address   Hex dump   Disassembly                  Comment
013C9B98  33C0       xor     eax,eax              DAP.0055CFE4
013C9B9A  EB 02      jmp     short 013C9B9E
013C9B9C  CD 20      int     20
013C9B9E  E8 8D000000 call   013C9C30
013C9BA3  C3         ret
013C9BA4  EB 04      jmp     short 013C9BAA
```

y ya podemos tracear con normalidad la zona sin peligro [por ahora] puesto que no sabemos todavía lo que podemos encontrar de aquí en adelante, seguimos traceando y mira por dónde! si antes lo digo antes pasa, grrr..

Resulta que no mucho más adelante salta a otra sección y mirad:

```
Address   Hex dump   Disassembly                  Comment
01400662  0F31       rdtsc                        ; mas rdtsc;(
01400664  EB 02      jmp     short 01400668
01400666  CD 20      int     20
01400668  E8 8D000000 call   014006FA
0140066D  C3         ret
0140066E  EB 04      jmp     short 01400674
```

Afortunadamente en esta sección hay muy pocos:

```
01400662   rdtsc          más rdtsc ;(
014006B7   rdtsc              ...
01400769   rdtsc              ...
014007BE   rdtsc              ...
```

Son solo cuatro, si traceamos un poco pasaremos por encima éstos, como ya sabemos para lo que están, simplemente los reemplazamos por "xor eax, eax" como antes y continuamos, esto lo podemos hacer a mano. El siguiente punto será encontrar el OEP, recordemos que antes llegamos a una zona que comenzaba con un PUSH 0 pero que después de estudiarla deducimos que no podía ser el OEP, así que veamos si lo podemos localizar ahora, volvemos a colocar el BPM ON MEMORY ACCESS en toda la sección de código y dejamos que el programa corra pulsando F9, para aquí:

```
Address   Hex dump   Disassembly                  Comment
00401000  55         push    ebp <------ El OEP?
00401001  8BEC       mov     ebp,esp
00401003  33C0       xor     eax,eax
00401005  50         push    eax
00401006  50         push    eax
00401007  50         push    eax
00401008  FF75 1C    push    dword ptr [ebp+1C]
0040100B  FF75 18    push    dword ptr [ebp+18]
0040100E  FF75 14    push    dword ptr [ebp+14]
00401011  FF75 10    push    dword ptr [ebp+10]
00401014  FF75 0C    push    dword ptr [ebp+C]
00401017  68 D8B65100 push    0051B6D8
0040101C  E8 FD8E0F00 call    004F9F1E
00401021  5D         pop     ebp
00401022  C2 1C00    ret     1C <------ Y esto?
00401025  B8 50BD5100 mov    eax,?messageMap@CSkinDlg@@1UAI
0040102A  C3         ret
```

Vaya, un PUSH EBP y en 401000, jeje… a priori tiene toda la pinta de OEP, de momento traceamos la zona con F7, al entrar en el call de :0040101C veo:

```
Address   Hex dump     Disassembly          Comment
004F9F1E  - FF25 80B05100  jmp     [51B080]
004F9F24  - FF25 88B05100  jmp     [51B088]
004F9F2A  - FF25 84B05100  jmp     [51B084]
004F9F30  - FF25 74B05100  jmp     [51B074]
004F9F36  - FF25 7CB05100  jmp     [51B07C]
004F9F3C  - FF25 78B05100  jmp     [51B078]
004F9F42  - FF25 68B05100  jmp     [51B068]
004F9F48  - FF25 70B05100  jmp     [51B070]
004F9F4E  - FF25 6CB05100  jmp     [51B06C]
004F9F54  - FF25 5CB05100  jmp     [51B05C]
004F9F5A  - FF25 64B05100  jmp     [51B064]
```

Vaya! se parece mucho a una Jmp_Table, pero… donde están las apis? No se ve ninguna, seguimos el primer JMP y se pone la pantalla nuevamente en blanco [petardazo] dejando al Olly mostrando un mensaje similar al anterior, hmmm…

Parece que estamos haciendo algo mal, grrr.. incluso hay veces en que no llega a parar en la sección de código y falla por un ACCESS_VIOLATION, no he encontrado el motivo del porqué pasa esto aunque hay veces que ocurre ¿?

Ante este escenario deberemos comenzar TODO de nuevo ;(Aunque esta vez trataremos únicamente de localizar el OEP, tampoco utilizaremos el método de las excepciones sino todo lo contrario, las desactivaremos tildándolas en la ventanita de Exceptions de entrada y colocaremos el BPM en la sección de código, Olly irá parando de vez en cuando por accesos de lectura y/o escritura, cada loop que aparezca lo desharemos poniendo un Bp con F2 al final del mismo, quitando antes el BPM de la sección de código para que no moleste y cuando para quitamos el Bp colocando de nuevo el BPM, esto lo iremos repitiendo para cada loop que aparezca, no son muchos, los más interesantes son estos que muestro:

```
Address   Hex dump     Disassembly          Comment
013CF14F    8A06       mov     al,[esi]     ; aqui para Olly
013CF151    46         inc     esi
013CF152    47         inc     edi
013CF153    8843 0F    mov     [ebx+F],al
013CF156    8A46 FF    mov     al,[esi-1]
013CF159    55         push    ebp
013CF15A    E8 00000000  call  013CF15F
013CF15F    5D         pop     ebp
013CF160    81ED 0D470000  sub ebp,470D
013CF166    8A8D 50030000  mov cl,[ebp+350]
013CF16C    5D         pop     ebp
013CF16D    32C1       xor     al,cl
013CF16F    8847 FF    mov     [edi-1],al
013CF172    8BC5       mov     eax,ebp
013CF174    4D         dec     ebp
013CF175    85C0       test    eax,eax
013CF177  ^ 75 A4      jnz     short 013CF11D
013CF179    33C0       xor     eax,eax      ; loop1 -> BP aqui..
013CF17B    5D         pop     ebp
```

Este es el primero en el que para, después llega a uno todavía mayor, pero lo traceamos y localizamos un punto de salida:

```
Address   Hex dump     Disassembly          Comment
013C9ACF    56         push    esi
013C9AD0    8BF7       mov     esi,edi
013C9AD2    2BF0       sub     esi,eax
013C9AD4    F3:A4      rep     movsb
013C9AD6    5E         pop     esi
013C9AD7  ^ E9 EFFEFFFF  jmp   013C99CB     ; fin Loop2
013C9ADC    8B75 08    mov     esi,[ebp+8]  ; Bp aqui..
013C9ADF    837E 06 00 cmp     dword ptr [esi+6],0
013C9AE3  v 74 46      je      short 013C9B2B
013C9AE5    56         push    esi
013C9AE6    8B4E 02    mov     ecx,[esi+2]
```

Ponemos un BP donde muestra la imagen, cuando pare estaremos cerca del RET que nos saca de la rutina, luego vienen un par de REP MOVS sin importancia que pasamos con F8 de un tirón y llegamos al primer RDTSC en :013C9B98, es el momento de colocar nuevamente el BPM en la sección de código y después de una paradita + F9 por fin encontramos lo que andamos buscando, el OEP¡!!!! jeje..

```
Address   Hex dump     Disassembly                          Comment
004FB4FA  90           nop
004FB4FB  90           nop
004FB4FC  90           nop                                  Stolen
004FB4FD  90           nop                                  Bytes!
004FB4FE  90           nop
004FB4FF  90           nop
004FB500  E8 43010000  call    004FB648
004FB505  391D 98575700 cmp    [575798],ebx
004FB50B  75 0C        jnz     short 004FB519
004FB50D  68 5E384A00  push    004A385E
004FB512  FF15 04B15100 call   [51B104]                     msvcrt.__setusermatherr
004FB518  59           pop     ecx
004FB519  E8 18010000  call    004FB636
004FB51E  68 74D15500  push    0055D174
004FB523  68 70D15500  push    0055D170
004FB528  E8 03010000  call    004FB630                     jmp to msvcrt._initterm
004FB52D  A1 98AD5700  mov     eax,[57AD98]
004FB532  8945 94      mov     [ebp-6C],eax
004FB535  8D45 94      lea     eax,[ebp-6C]
004FB538  50           push    eax
004FB539  FF35 94AD5700 push   dword ptr [57AD94]
004FB53F  8D45 9C      lea     eax,[ebp-64]
004FB542  50           push    eax
004FB543  8D45 90      lea     eax,[ebp-70]
004FB546  50           push    eax
004FB547  8D45 A0      lea     eax,[ebp-60]
004FB54A  50           push    eax
004FB54B  FF15 C4B05100 call   [51B0C4]                     msvcrt.__getmainargs
```

Esto si se parece más a un OEP, además aquí si que encontramos lo stolen bytes que sabemos que inserta este packer, si miramos la pila vemos que hay muchos:

```
Address   Value    Comment
0012FF3C  77D121AE USER32.77D121AE
0012FF40  0012CD34
0012FF44  7FFDF000
0012FF48  02E8E800
0012FF4C  CD000000
0012FF50  24048320
0012FF54  2444830B
0012FF58  E9C31304
0012FF5C  0002E849
0012FF60  20CD0000
0012FF64  08240483
0012FF68  E1FFE8C3
0012FF6C  000001E8
0012FF70  C4839A00
0012FF74  E8E0FF04
0012FF78  0012FF35
0012FF7C  77D121AE USER32.77D121AE
0012FF80  0012CD34
0012FF84  0012FE3F
0012FF88  0012FF9C
0012FF8C  7FFDF000
0012FF90  000015F7
0012FF94  0012FE84 RETURN to 0012FE84 from B917C307
0012FF98  0012FF5A
0012FF9C  0012FEAE RETURN to 0012FEAE from 0012FEAF
0012FFA0  0012FEE9 RETURN to 0012FEE9 from 0012FEEC
0012FFA4  015705B3
0012FFA8  0012FF3C
0012FFAC  B1478E9A
0012FFB0  0012FFE0 Pointer to next SEH record
0012FFB4  004FB61E SE handler
0012FFB8  00530CF0 DAP.00530CF0
0012FFBC  00000000
0012FFC0  0012FFF0
0012FFC4  77E614C7 RETURN to kernel32.77E614C7
0012FFC8  77D121AE USER32.77D121AE
0012FFCC  0012CD34
```

Son bastantes, y habrá que reconstruirlos para que corra el dumpeado, ya veremos después cómo hacerlo, de momento delimitaremos la iat ya que estamos, este paso no lo detallaré pues está más que explicado en otros tutes:

```
Address   Hex dump                                         ASCII
0051A000  BA 86 57 01 C8 86 57 01 CD 86 57 01 DB 86 57 01
0051A010  E9 86 57 01 FB 86 57 01 09 87 57 01 1B 87 57 01
0051A020  20 87 57 01 2E 87 57 01 3C 87 57 01 41 87 57 01
0051A030  4F 87 57 01 5D 87 57 01 6B 87 57 01 79 87 57 01
0051A040  00 00 00 00 0E 78 57 01 1C 78 57 01 32 78 57 01
0051A050  44 78 57 01 49 78 57 01 5B 78 57 01 DC CC 09 78
0051A060  69 78 57 01 7B 78 57 01 8D 78 57 01 9F 78 57 01
0051A070  B1 78 57 01 00 00 00 00 06 84 57 01 10 84 57 01
0051A080  1E 84 57 01 23 84 57 01 35 84 57 01 43 84 57 01
0051A090  50 84 57 01 62 84 57 01 78 84 57 01 7D 84 57 01
0051A0A0  8F 84 57 01 94 84 57 01 A9 84 57 01 B7 84 57 01
0051A0B0  C9 84 57 01 DF 84 57 01 F1 84 57 01 03 85 57 01
```

Inicio: 51A000

```
Address   Hex dump                                          ASCII
0051B540  F0 76 57 01 FE 76 57 01 0C 77 57 01 1A 77 57 01  ðvW.þvW..wW..wW.
0051B550  00 00 00 00 E2 5D A3 71 8D 86 A3 71 BF 2B A3 71  ....à]£q¢†£q¿+£q
0051B560  0D 35 A3 71 40 17 A3 71 36 18 A3 71 DA 41 A3 71  .5£q@↨£q6↑£qÚA£q
0051B570  F8 12 A3 71 00 00 00 00 AD 86 57 01 00 00 00 00  ø↕£q....-†W.....
0051B580  F5 87 57 01 07 88 57 01 15 88 57 01 1E 88 57 01  õ‡W.•ˆW.§ˆW.ˆW.
0051B590  00 00 00 00 E0 58 00 10 80 1A 00 10 60 1A 00 10  ....àX.€→.→`→.→
0051B5A0  90 58 00 10 00 5A 00 10 00 00 00 00 00 00 00 00  ÊX.►.Z.►........
0051B5B0  25 10 40 00 B8 B5 51 00 13 01 00 00 00 00 00 00  %►@.¸µQ.‼.......
```

Final: 51B5A8

Restando ambos obtenemos su longitud:

Largo: 15A8h

Vemos además que hay muchos punteros malos que apuntan a direcciones que están fuera del ejecutable del tipo :0157xxxx en mi máquina.

Pues éste será el siguiente paso, trataremos de evitar que se redireccione parte de la iat, nos centramos en el primer puntero [51A000]==015786BAh, trabajaremos con un BPM en la entrada porque no responde a los HBPs, reinicio Olly y le pongo uno en escritura (BPM MEMORY - ON WRITE) a la primera va la vencida ya que nos deja en la zona caliente:

```
Address   Hex dump        Disassembly                          Comment
013C61CB    8943 01       mov     [ebx+1],eax
013C61CE    0111          add     [ecx],edx
013C61D0    5F            pop     edi
013C61D1    58            pop     eax
013C61D2    8907          mov     [edi],eax                    ; rellena lat IAT
013C61D4    61            popad
013C61D5    8385 43010200 add     dword ptr [ebp+20143],4
013C61DC  ^ E9 B0FBFFFF   jmp     013C5D91
```

Si miramos los registros encontramos lo que veníamos buscando:

```
Registers (FPU)
EAX 015786BA  <---- Bytes a salvar
ECX 013C0384
EDX 00000005
EBX 015786C3
ESP 0012E034
EBP 013A0000  <---- Base sección temporal
ESI 013C038C
EDI 0051A000 DAP.0051A000  <---- Puntero a la IAT
EIP 013C61D2
```

Lo tenemos prácticamente TODO en los registros, desde la base de la sección donde apuntan las entradas hasta la dirección de la iat con la que trabaja, así como los bytes finales de dicha entrada, traceamos con cuidado esta zona, nada más tocar F7 ejecuta un POPAD que pone en EAX una api:

```
Registers (FPU)
EAX 77DA28BB ADVAPI32.RegCreateKeyA  <---- Una API ;)
ECX 77E5B5E1 kernel32.77E5B5E1
EDX 00400000 DAP.00400000
EBX 0055866C DAP.0055866C
ESP 0012E054
EBP 013A0000
ESI 00555694 DAP.00555694
EDI 0051A000 DAP.0051A000
EIP 013C61D5
```

Aquí ya podríamos intentar algo pero seremos pacientes, jeje.. traceamos esta zona y vemos que nos encontramos en un loop que va desde:

```
Address   Hex dump        Disassembly                          Comment
013C5D91    8B06          mov     eax,[esi]                    ; inicio Loop..
013C5D93    0BC0          or      eax,eax
013C5D95    8B55 00       mov     edx,[ebp]
013C5D98  v 75 03         jnz     short 013C5D9D
013C5D9A    8B46 10       mov     eax,[esi+10]
013C5D9D    03C2          add     eax,edx
013C5D9F    0385 43010200 add     eax,[ebp+20143]
```

Que va hasta:

```
Address   Hex dump        Disassembly                          Comment
013C61CE  0111            add     [ecx],edx
013C61D0  5F              pop     edi
013C61D1  58              pop     eax
013C61D2  8907            mov     [edi],eax                    ; rellena lat IAT
013C61D4  61              popad
013C61D5  8385 43010200   add     dword ptr [ebp+20143],4
013C61DC  E9 B0FBFFFF     jmp     013C5D91                     ; Fin Loop..
013C61E1  83C6 14         add     esi,14
```

Que es donde rellena la tabla, cuando termine el trabajo con ésta saldrá de estos límites y dejará de loopear, por ahora no lo hace y podemos tracear la zona tranquilamente con F7, enseguida encontramos unos extraños saltos que pasamos sin que se ejecute ninguno, luego nos pone la base de una DLL en EDX:

```
Address   Hex dump        Disassembly                          Comment
013C5E8B  813B 95B75126   cmp     dword ptr [ebx],2651B795
013C5E91  0F84 102A0000   je      013C88A7
013C5E97  813B B482F64B   cmp     dword ptr [ebx],4BF682B4
013C5E9D  0F84 552A0000   je      013C88F8
013C5EA3  813B 0F1ACF4C   cmp     dword ptr [ebx],4CCF1A0F
013C5EA9  0F84 942A0000   je      013C8943
013C5EAF  813B 4A7687DF   cmp     dword ptr [ebx],DF87764A
013C5EB5  0F84 F92D0000   je      013C8CB4
013C5EBB  813B B8B8B2FB   cmp     dword ptr [ebx],FBB2B8B8
013C5EC1  0F84 54340000   je      013C931B
013C5EC7  813B 8E5D2D57   cmp     dword ptr [ebx],572D5D8E
013C5ECD  0F84 84340000   je      013C9357
013C5ED3  60              pushad
013C5ED4  8B03            mov     eax,[ebx]
013C5ED6  8985 5E5C0200   mov     [ebp+25C5E],eax
013C5EDC  8B95 2F010200   mov     edx,[ebp+2012F]
013C5EE2  8B42 3C         mov     eax,[edx+3C]
013C5EE5  03C2            add     eax,edx                      ADVAPI32.77DA0000
013C5EE7  8B40 78         mov     eax,[eax+78]
```

Le añade este valor a EAX que valía 0x108h quedando EAX==77DA0108h que apunta a la cadena "PE" que cae dentro de la cabecera de la librería Advapi32.dll, luego llegamos a este call:

```
013C5F01    8B36            mov     esi,[esi]
013C5F03    03F2            add     esi,edx
013C5F05    E8 D4140000     call    013C73DE
013C5F0A    5E              pop     esi
013C5F0B    50              push    eax
```

Y ya vemos una api en el registro ESI:

```
Registers (FPU)
ESP 0012E030
EBP 013A0000
ESI 77E00E53 ASCII "A_SHAFinal"  <---  Una API ;)
EDI 0051A004 DAP.0051A004
EIP 013C5F05
```

Pero todavía no es la definitiva pues está en otro bucle [si uno más XDD] en el que parece buscar la api que necesita:

```
Address   Hex dump        Disassembly                          Comment
013C5EFC  83C6 04         add     esi,4
013C5EFF  43              inc     ebx
013C5F00  56              push    esi
013C5F01  8B36            mov     esi,[esi]
013C5F03  03F2            add     esi,edx                      ; la API en ESI
013C5F05  E8 D4140000     call    013C73DE
013C5F0A  5E              pop     esi
013C5F0B  50              push    eax
013C5F0C  8B85 BA5B0200   mov     eax,[ebp+25BBA]
013C5F12  3985 5E5C0200   cmp     [ebp+25C5E],eax
013C5F18  58              pop     eax
013C5F19  75 E1           jnz     short 013C5EFC               ; Fin Loop..
013C5F1B  8B48 24         mov     ecx,[eax+24]
```

Se supone que cuando la encuentre dejará de loopear, así pues ponemos un BP justo después del JNZ [de :013C5F1B] y pulsamos F9, paramos allí y traceamos un poco hasta ver qué api buscaba.

En este caso la encontramos en :013C5F38 en ECX:

```
Registers (FPU)
EAX 77DFF3D4 ADVAPI32.77DFF3D4
ECX 77DA2FCD ADVAPI32.OpenProcessToken
EDX 77DA0000 ADVAPI32.77DA0000
EBX 000001A8
ESP 0013F034
```

Ahora falta saber si es la api definitiva que debería ir en la iat, seguimos traceando y vemos que pasa la api de ECX a EAX, después llegamos a esta zona:

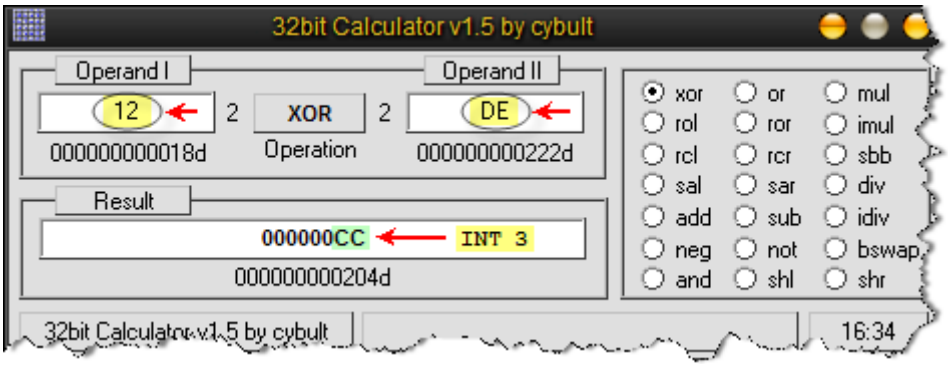
```
Address   Hex dump        Disassembly                          Comment
013C6104  0000           add     [eax],al
013C6106  0000           add     [eax],al
013C6108  80BD B85B0200 0 cmp     byte ptr [ebp+25BB8],1       ; Revisa un flag..
013C610F  60             pushad
013C6110  0F84 BC000000  je      013C61D2                     ; ojo con el salto!
013C6116  8B8D 5B010200  mov     ecx,[ebp+2015B]
013C611C  8D99 BF760200  lea     ebx,[ecx+276BF]
013C6122  8D8D 84030200  lea     ecx,[ebp+20384]
```

Si miramos la ventanita auxiliar vemos que lee de [013C5BB8] cuyo byte bajo está a 0, de momento NO va a saltar y llegamos a una interesante zona:

```
Address   Hex dump   Disassembly                 Comment
013C6139  8A10       mov     dl,[eax]            ; lee primer byte de la api
013C613B  50         push    eax                 ; empuja la api al stack
013C613C  B0 12      mov     al,12               ; Modifica su direccion en EAX
013C613E  32C2       xor     al,dl               / Mira si hay un
013C6140  34 DE      xor     al,0DE              ; BP puesto de
013C6142  75 03      jnz     short 013C6147      \ manera disimulada..
013C6144  80C2 01    add     dl,1                ; si lo hay llegamos aqui ;(
013C6147  58         pop     eax
013C6148  80FA 50    cmp     dl,50
```

Donde hace dos cosas, por un lado salva la api en la pila en el PUSH EAX y por último revisa que no haya un BP en el primer byte de la misma, lo hace de un modo muy sutil aunque darse cuenta es fácil, veamos.. coloca el valor 0x12h en AL que XORea con el primer byte de la api, el resultado lo XORea con 0xDEh después viene un salto condicional del tipo JNZ, es muy importante que salte como se ve en la imagen pues de no hacerlo querrá decir que ha detectado un BP [INT 3] en la primera línea de la api tratada, pero… Cómo lo sabemos?

Cojamos la ***calculadora de cybult*** y carguemos esos dos valores en los campos Operando1 y Operando2 haciendo con ellos un XOR, o sea esto:



Allí se ve con más claridad, el Operando1==12h y el Op2==0Deh, que XOReados entre sí dan como resultado 0xCCh que es el opcode del INT3, somos sabedores de que al colocar un BP con F2 internamente el debugger escribe el valor 0xCCh en el código reemplazando el byte original, lo que hace el packer es XORear el byte encontrado con 12h y el resultado con 0Deh, lo que equivale a XORear directamente con 0CCh, o sea que busca un BreakPoint en dicha instrucción, si encontrara una ejecutaría el "add dl,1" que nos descuadraría la posterior operación sin decirnos absolutamente NADA, jeje… qué pillín XDD

Dicho esto prosigamos con la rutina, sigue comparando el resultado obtenido en DL con ciertos valores constantes tipo 50, 5F, 6A, 68, etc., no coincide con

ninguno, entra en un bucle que se repite 5 veces y después viene lo más interesante de esta rutina, es esto:

```
Address   Hex dump        Disassembly                      Comment
013C61C0  59              pop     ecx
013C61C1  6A 05           push    5                        ; / Aqui llegamos con un
013C61C3  5A              pop     edx                      ; \ valor calculado en EBX
013C61C4  2BC3            sub     eax,ebx                  ; lo resta a EAX [tiene la API]
013C61C6  2BC2            sub     eax,edx                  ; le resta 5 bytes mas
013C61C8  C603 E9         mov     byte ptr [ebx],0E9       ; inserta un JMP
013C61CB  8943 01         mov     [ebx+1],eax              ; y lo reajusta..
013C61CE  0111            add     [ecx],edx
013C61D0  5F              pop     edi
013C61D1  58              pop     eax
013C61D2  8907            mov     [edi],eax                ; rellena lat IAT
013C61D4  61              popad
013C61D5  8385 43010200   add     dword ptr [ebp+20143],4
013C61DC  E9 B0FBFFFF     jmp     013C5D91                 ; Fin Loop..
```

Después de estas operaciones deja EAX==015786C8h valor que depositará en la iat en la entrada [51A004], y resulta que en esa dirección [013C61C8] acaba de escribir código, allí donde dice "inserta un JMP" y "lo reajusta" está escribiendo en esa zona y esto es lo que dejó XDD:

```
Address   Hex dump        Disassembly                      Comment
015786C1  8BEC            mov     ebp,esp
015786C3  - E9 F6A18276   jmp     ADVAPI32.77DA28BE
015786C8  - E9 00A98276   jmp     ADVAPI32.OpenProcessToken
015786CD  0000            add     [eax],al
015786CF  0000            add     [eax],al
015786D1  0000            add     [eax],al
015786D3  0000            add     [eax],al
015786D5  0000            add     [eax],al
```

Insertó un JMP directo a la api, de esto concluimos que la api que vemos en la dirección :013C5F38 en ECX es la correcta para esa entrada, además si el salto que hay en :013C6110 se ejecutara llegaríamos directamente a la zona donde guarda la api sin pasar por la parte que manipula sus bytes y la redirecciona, o sea que es el MÁGICO ;)

```
Address   Hex dump        Disassembly                      Comment
013C6106  0000            add     [eax],al
013C6108  80BD B85B0200   cmp     byte ptr [ebp+25BB8],1   ; Revisa un flag..
013C610F  60              pushad
013C6110  0F84 BC000000   je      013C61D2                 ; SALTO MAGICO ;)
013C6116  8B8D 5B010200   mov     ecx,[ebp+2015B]
013C611C  8D99 BF760200   lea     ebx,[ecx+276BF]
```

Por su puesto nos anotamos esta dirección para luego:

Salto Mágico: 013C6110

JMPeando este salto llegamos al OEP con algunas entradas todavía redirigidas, esto indica que debe tener algún otro lugar donde rellena la tabla, veamos dónde. La manera de encontrar esta parte es sencilla, colocando un BPM en una cualquiera de las entradas que quedaron mal, en total son 8:

Entrada	IAT rva:	Apunta a:
1.-	11A15C	01569B75
2.-	11A168	0155AE6C
3.-	11A24C	01559E56
4.-	11A2B0	0156AB8B
5.-	11B28C	015664E6
6.-	11B290	01568B53
7.-	11B294	01561E99
8.-	11B430	0156BC35

Tenemos donde elegir, iremos por el menor que es [51A15C] recordemos que la dirección la obtenemos mediante la fórmula "rva + imagebase" reiniciamos Olly como siempre con F2 y ponemos un BPM en la dirección de esta entrada y vemos donde escribe el valor final de la misma:

```
Address   Hex dump   Disassembly                    Comment
013C8C8D  EB 02      jmp    short 013C8C91
013C8C8F  CD 20      int    20
013C8C91  890F       mov    [edi],ecx               ; rellena [51A15C]
013C8C93  7C 03      jl     short 013C8C98
013C8C95  EB 03      jmp    short 013C8C9A
```

Aquí escribe 01569B75 que será el valor definitivo, ahora debemos mirar a ver si encontramos algún mágico como hicimos con las otras entradas, Reiniciamos Olly, traceamos el loop donde rellena la tabla y encontramos algo:

```
Address   Hex dump   Disassembly                    Comment
013C5E8B  813B 95B75126  cmp    dword ptr [ebx],2651B795
013C5E91  0F84 102A0000  je     013C88A7
013C5E97  813B B482F64B  cmp    dword ptr [ebx],4BF682B4
013C5E9D  0F84 552A0000  je     013C88F8
013C5EA3  813B 0F1ACF4C  cmp    dword ptr [ebx],4CCF1A0F  ; Salta para [51A15C]
013C5EA9  0F84 942A0000  je     013C8943
013C5EAF  813B 4A7687DF  cmp    dword ptr [ebx],DF87764A
013C5EB5  0F84 F92D0000  je     013C8CB4
013C5EBB  813B B8B8B2FB  cmp    dword ptr [ebx],FBB2B8B8
013C5EC1  0F84 54340000  je     013C931B
013C5EC7  813B 8E5D2D57  cmp    dword ptr [ebx],572D5D8E
```

Ese JE antes NO se ejecutaba y ahora sí que lo hace cuando trata de rellenar dicha entrada de la IAT, si sigo el salto y traceo la zona al llegar aquí:

```
Address   Hex dump   Disassembly                    Comment
014005B8  0000       add    [eax],al
014005BA  00E8       add    al,ch
014005BC  83C4 04    add    esp,4
014005BF  0337       add    esi,[edi]               ; Recoge la Api ;)
014005C1  EB 02      jmp    short 014005C5
014005C3  CD 20      int    20
```

Recoge una Api que es dejada en ESI como podemos ver fácilmente:

```
Registers (FPU)
ESP 0012E028
EBP 01400000
ESI 77E5D142 kernel32.GetVersion  <---  Api válida ;)
EDI 013A214C
EIP 014005BF
```

Se trata de GetVersion, jeje.. de esta misma forma podemos obtener el nombre de otras apis que el packer trata de diferente manera. Podemos reafirmar esto traceando nuevamente hasta que vuelva a saltar en uno de los saltos:

```
Address   Hex dump   Disassembly                    Comment
013C5E13  813B 9404B2D9  cmp    dword ptr [ebx],D9B20494
013C5E19  0F84 A71C0000  je     013C7AC6
013C5E1F  813B A41A86D0  cmp    dword ptr [ebx],D0861AA4
013C5E25  0F84 55210000  je     013C7F80
013C5E2B  813B 706586B1  cmp    dword ptr [ebx],B1866570
013C5E31  0F84 BE240000  je     013C82F5                ; Salta para [51A168]
013C5E37  813B 0E46769B  cmp    dword ptr [ebx],9B76460E
013C5E3D  0F84 33280000  je     013C8676
013C5E43  813B DB0793E6  cmp    dword ptr [ebx],E69307DB
013C5E49  0F84 73280000  je     013C86C2
013C5E4F  813B 627B6CA5  cmp    dword ptr [ebx],A56C7B62
```

Ahora lo hace para la entrada [51A168] que es la siguiente a rellenar, está leyendo de [EBX] donde tiene almacenado el valor 0x4CCF1A0Fh el cual compara, si salta nos lleva a una zona similar a la anterior pero algo distinta donde rellena la entrada con el valor que le da la gana, si no dejamos que salte cambiando el flag Z [Zero] llegamos al loop donde busca la api:

```
Address   Hex dump   Disassembly                    Comment
013C5EFC  83C6 04    add    esi,4
013C5EFF  43         inc    ebx
013C5F00  56         push   esi
013C5F01  8B36       mov    esi,[esi]
013C5F03  03F2       add    esi,edx                 ; nombre Api en ESI
013C5F05  E8 D4140000  call 013C73DE
013C5F0A  5E         pop    esi
013C5F0B  50         push   eax
013C5F0C  8B85 BA5B0200  mov  eax,[ebp+25BBA]
013C5F12  3985 5E5C0200  cmp  [ebp+25C5E],eax
013C5F18  58         pop    eax
013C5F19  75 E1      jnz    short 013C5EFC          ; FinLoop..
013C5F1B  8B48 24    mov    ecx,[eax+24]
```

Al salir del mismo traceo un pelín y veo otra vez la api en ECX, jeje..

```
Registers (FPU)
EAX 77EAD040 kernel32.77EAD040
ECX 77E5AD86 kernel32.GetModuleHandleA  <---  Api válida ;)
EDX 77E40000 kernel32.77E40000
EBX 0000016E
ESP 0012E034
EBP 013A0000
```

Era GetModuleHandleA, de esta manera podemos sacar algunas más en total estas:

Entrada:	Dirección:	Api válida
1.-	51A15C	GetVersion
2.-	51A168	GetModuleHandleA
3.-	51A24C	GetCurrentProcess
4.-	51A2B0	GetVersionExA

Pero aún quedan otras cuatro por resolver que aún traceando la zona donde rellena la tabla solamente conseguimos ver una constante que introduce en la IAT que es la dirección donde emulará la api de turno. Faltan estas:

Entrada	IAT rva:	Apunta a:
5.-	51B28C	015664E6
6.-	51B290	01568B53
7.-	51B294	01561E99
8.-	51B430	0156BC35

Incluso si le seguimos la pista a la entrada [51B430] vemos que salta aquí:

```
Address   Hex dump     Disassembly                          Comment
013C5E9D  0F84 552A0000  je      013C88F8
013C5EA3  813B 0F1ACF4C  cmp     dword ptr [ebx],4CCF1A0F
013C5EA9  0F84 942A0000  je      013C8943
013C5EAF  813B 4A7687DF  cmp     dword ptr [ebx],DF87764A
013C5EB5  0F84 F92D0000  je      013C8CB4
013C5EBB  813B B8B8B2FB  cmp     dword ptr [ebx],FBB2B8B8
013C5EC1  0F84 54340000  je      013C931B
013C5EC7  813B 8E5D2D57  cmp     dword ptr [ebx],572D5D8E
013C5ECD  0F84 84340000  je      013C9357                     ; caso [51B430]
013C5ED3  60             pushad
```

Si seguimos el salto llegamos a esta parte:

```
013C937B    FFE0            jmp     eax
013C937D    61              popad
013C937E    0385 4C210000   add     eax,[ebp+214C]
013C9384    8981 47CC0100   mov     [ecx+1CC47],eax
013C938A    8D89 35BC0100   lea     ecx,[ecx+1BC35]
013C9390    890F            mov     [edi],ecx
013C9392    61              popad
013C9393  ^ E9 3DCEFFFF     jmp     013C61D5
```

Si miramos ahora la ventana de registros vemos:

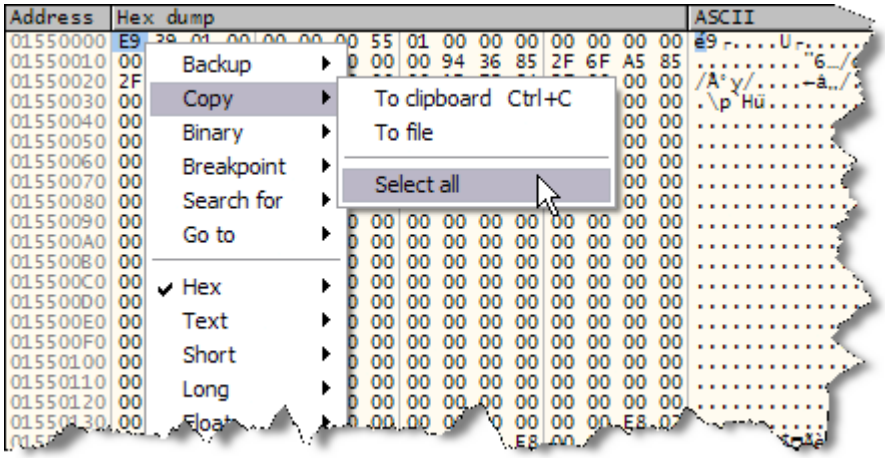
```
Registers (MMX)
EAX 77D3E824 USER32.MessageBoxA  <---  ¿Es la Api válida?
ECX 0156BC35
EDX 00400000 DAP.00400000
EBX 00557B4C DAP.00557B4C
ESP 0012E034
EBP 013A0000
```

Pero si reparamos esta entrada como MessageBoxA estaremos cometiendo un grave error porque no se corresponde con esta Api, imagino que esto la hace para despistar todavía más.

Lo que nos quedaría por hacer sería injertar la sección :01550000 donde ejecuta las funciones Api en nuestro dumpeado, y después con VirtualAlloc [VAlloc de ahora en adelante] insertarla en el dumpeado.

Nuestra amiga VirtualAlloc ;)

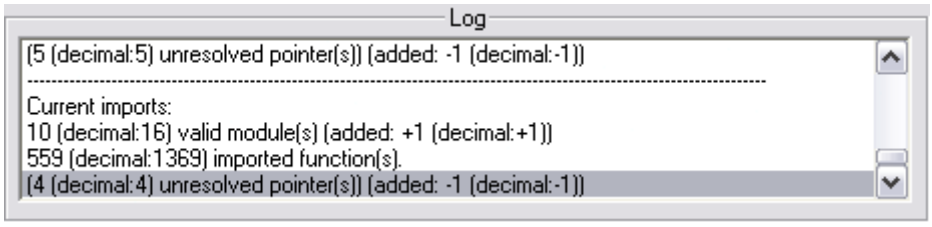
Pues sí, así es y visto que no hay rastro de Api en las 4 entradas restantes de la iat llega el momento de injertar. Primero salvamos la sección necesaria a un lugar seguro, lo haremos desde el OEP mejor:



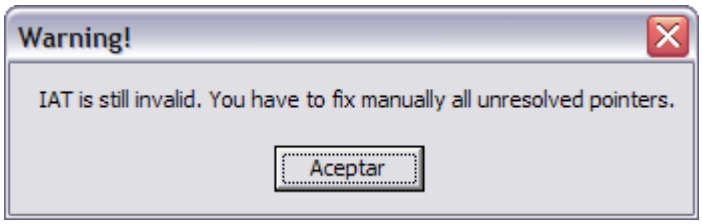
Después hacemos un BINARY COPY y salvamos su contenido en un archivo de texto, así de fácil, y ya de paso dumpeamos puesto que estamos en el OEP, utilizamos las opciones comunes: destildamos REBUILD IMPORT y picamos GET EIP AS OEP una vez, luego DUMP y lo llamamos "tute.exe" por ejemplo.

Una vez dumpeado abrimos el ImpREC v1.6f y le damos el OEP correcto 0x0FB500h que en realidad es el offset del mismo, picamos IAT AUTOSEARCH para que saque los valores de tabla [que ya conocemos] como los da bien no los tocamos, luego a GET IMPORTS, hay que decir que para que obtenga las entradas en condiciones deberemos haber JMPeado el MÁGICO:

Pues si no hacemos esto quedarán más de 300 entradas por resolver, además reparamos las 4 que ya conocemos [Tabla anterior] y quedarán otras 4 por resolver como en la siguiente imagen:



Son tres de la misma DLL y otra del USER, pulsamos FIX DUMP y nos dice esto:



O sea, que quedaron entradas por reparar, como ya sabíamos, pero esto tiene un inconveniente, por las 3 de la misma DLL no hay problema, pero... la del USER…

nos perjudica bastante, grrr.. echemos un vistazo a la tabla:

Address	Hex dump	ASCII
0051B1C4	00 36 C1 77 70 BD BE 77 C0 36 C1 77 7E FE BF 77	.6Áwp½%wÀ6Áw~þ¿
0051B1D4	F5 19 BF 77 75 00 C0 77 85 BF C0 77 60 2E C1 77	õ.¿wu.Àw…¿Àw`.Á
0051B1E4	75 D9 C0 77 92 C8 C0 77 55 C0 BE 77 80 10 C0 77	uÙÀw'ÈÀwUÀ%w€.À
0051B1F4	74 8E C1 77 A4 87 C1 77 7F C0 BE 77 70 BA C1 77	tŽÁw¤‡ÁwÀ%wpºÁ
0051B204	C8 4C C0 77 80 8A C1 77 8D BB C1 77 3E 05 C0 77	ÈLÀw€ŠÁw»Áw>.À
0051B214	3F E9 C0 77 21 D3 BE 77 D0 8B C1 77 A3 43 C0 77	?éÀw!Ó%wÐ‹Áw£CÀ
0051B224	13 67 C1 77 79 6B C1 77 00 00 00 00 4B 17 0F 77	.gÁwykÁw....K.w
0051B234	44 16 0F 77 56 38 10 77 74 16 0F 77 EC 14 0F 77	D.wV8.wt.wì.w
0051B244	8F 3D 0F 77 7E 36 0F 77 FD 36 0F 77 DD F7 0F 77	.=w~6wý6wÝ÷w
0051B254	50 16 0F 77 00 00 00 00 66 E5 64 00 5E 7D 6E 00	P.w....fåd.^}n.
0051B264	97 44 67 00 B2 E6 63 00 50 98 72 00 89 48 67 00	—Dg.²æc.P˜r.‰Hg.
0051B274	96 74 6F [illegible] 5 USER32.DLL [illegible] 67 00 00 00 00 00	–tn.µ5g.C?g....
0051B284	CC 5E [illegible] 00 00 00 00 E6 64 56 01 53 8B 56 01	Ì^.w....ædV.S‹V.
0051B294	99 1F [illegible] 01 00 00 00 00 A0 7D 57 01 A5 7D 57 01	™.V.... }W.¥}W.
0051B2A4	AA 7D 57 01 B8 7D 57 01 C6 7D 57 01 D4 7D 57 01	ª}W.¸}W.Æ}W.Ô}W.
0051B2B4	F3 7D 57 01 16 7E 57 01 1B 7E 57 01 20 7E 57 01	ó}W.–~W.›~W. ~W.
0051B2C4	2E 7E 57 01 33 7E 57 01 45 7E 57 01 4A 7E 57 01	.~W.3~W.E~W.J~W.
0051B2D4	54 7E 57 01 62 7E 57 01 67 7E 57 01 6C 7E 57 01	T~W.b~W.g~W.l~W.
0051B2E4	71 7E 57 01 76 7E 57 01 80 7E 57 01 89 7E 57 01	q~W.v~W.€~W.‰~W.

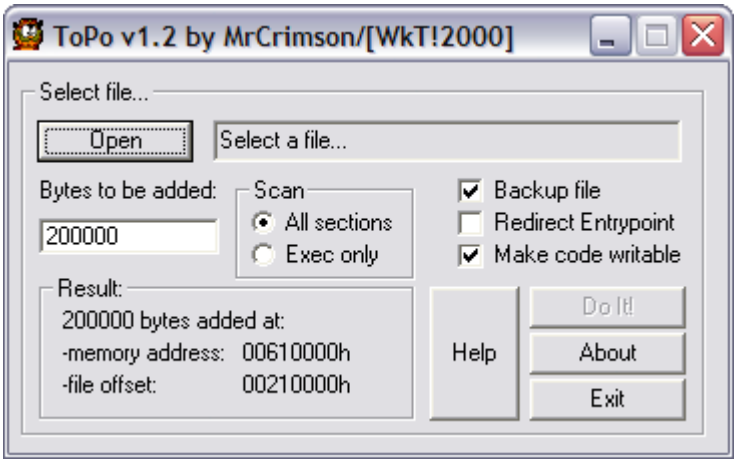
Resulta que no reparó el resto de entradas dejando todo el bloque inválido, esto tiene una fácil solución, hacemos CUT THUNK de la entrada y después la rellenamos en el injerto que haremos.

Address	Hex dump	ASCII
0051B400	34 0B D2 77 9C C8 D1 77 D9 88 D2 77 F4 8D D1 77	4.ÒwœÈÑwÙˆÒwôÑw
0051B410	F2 C2 D1 77 67 DA D2 77 20 D2 D1 77 20 D2 D1 77	òÂÑwgÚÒw ÒÑw ÒÑw
0051B420	7D C2 D1 77 B5 AC D1 77 C2 9F D1 77 02 8E D1 77	}ÂÑwµ¬ÑwÂŸÑw.ŽÑw
0051B430	00 00 00 00 06 70 D3 77 82 92 D1 77 52 A1 D1 77	.....pÓw‚'ÑwR¡Ñw
0051B440	EB DF D2 77 76 A3 D1 77 B3 9F D1 77 A1 EC D1 77	ëßÒwv£Ñw³ŸÑw¡ìÑw
0051B450	8A 9F D1 77 18 92 D1 77 37 DE D2 77 FF E8 D1 77	ŠŸÑw.'Ñw7ÞÒwÿèÑw
0051B460	53 AE D1 77 02 BC D1 77 23 9F D1 77 D6 8D D1 77	S®Ñw.¼Ñw#ŸÑwÖ.Ñw

Ahora reparó todas excepto la que hemos "cortado", lo siguiente es agregarle una nueva sección, podemos hacerlo a mano o bien utilizar alguna tool, por comodidad utilizaremos ToPo v1.2, excelente herramienta, la sección a añadir tiene un tamaño 0x2B000h bytes [≈175.000] como podemos ver:

Address	Size	Owner	Section	Contains	Type	Access	Init
00EF0000	00103000				Map	R	R
01000000	000BB000				Map	R E	R E
01310000	00002000				Map	R	R
01550000	0002B000				Priv	RW	RW
01580000	00001000				Priv	RW	RW
01690000	00002000				Map	R	R
01B10000	00002000				Map	R	R
10000000	00001000	ZLIB		PE header	Imag	R	RWE
[illegible]	[illegible]	ZLIB	[illegible]	[illegible]	[illegible]	[illegible]	RWE

Con una calculadora observamos que son exactamente 176.128 bytes, y por si las moscas injertaremos también el espacio de pila de tamaño 5000h bytes, por lo que la nueva sección agregada tendrá un tamaño de aproximadamente 200.000 bytes para que entren ambas secciones más el injerto propio:



Quedó a partir de :610000 con la técnica del BINARY COPY y BINARY PASTE copiamos los bytes de la sección :01550000 que tenemos en el archivo de texto y los pasamos allí, el final quedará a esta altura:

Address	Hex dump	ASCII
0063AF80	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	.....
0063AF90	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	.....
0063AFA0	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	.....
0063AFB0	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	.....
0063AFC0	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	.....
0063AFD0	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	.....
0063AFE0	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	.....
0063AFF0	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	.....
0063B000	90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90	ÉÉÉÉÉÉÉÉÉÉÉÉÉÉÉÉ
0063B010	90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90	ÉÉÉÉÉÉÉÉÉÉÉÉÉÉÉÉ
0063B020	90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90	ÉÉÉÉÉÉÉÉÉÉÉÉÉÉÉÉ
0063B030	90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90	ÉÉÉÉÉÉÉÉÉÉÉÉÉÉÉÉ
0063B040	90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90	ÉÉÉÉÉÉÉÉÉÉÉÉÉÉÉÉ

Hacemos lo mismo con la pila que tenemos aquí en el original:

Address	Size	Owner	Section	Contains	Type	Access	Init
00010000	00001000				Priv	RW	RW
00020000	00001000				Priv	RW	RW
0012A000	00001000				Priv	RW Gua	RW
0012B000	00005000			stack of ma	Priv	RW Gua	RW
00130000	00001000				Map	R	R
00140000	00002000				Map	R	R
00150000	00012000				Priv	RW	RW
00250000	00006000				Priv	RW	RW
0026[illegible]00	00001000				Map		RW

Pegamos los bytes justo debajo, a partir de :0063B000 en adelante, termina en:

Address	Hex dump	ASCII
0063FF80	34 CD 12 00 3F FE 12 00 9C FF 12 00 00 F0 FD 7F	4Í↕.?þ↕.œÿ↕..ðý
0063FF90	32 12 00 00 84 FE 12 00 5A FF 12 00 AE FE 12 00	2↕...„þ↕.Zÿ↕.®þ↕
0063FFA0	E9 FE 12 00 B3 05 57 01 3C FF 12 00 9A 8E 47 B1	éþ↕.³♣W☺<ÿ↕.šŽG±
0063FFB0	E0 FF 12 00 1E B6 4F 00 F0 0C 53 00 00 00 00 00	àÿ↕.¶O.ð♀S.....
0063FFC0	F0 FF 12 00 C7 14 E6 77 AE 21 D1 77 34 CD 12 00	ðÿ↕.Ç¶æw®!Ñw4Í↕.
0063FFD0	00 F0 FD 7F F0 AC E0 F6 C8 FF 12 00 4F D1 53 80	.ðý⌂ð¬àöÈÿ↕.OÑS€
0063FFE0	FF FF FF FF 09 48 E7 77 10 12 E7 77 00 00 00 00	ÿÿÿÿ.Hçw►↕çw....
0063FFF0	00 00 00 00 00 00 00 00 00 E0 5E 00 00 00 00 00	.........à^.....
00640000	90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90	ÉÉÉÉÉÉÉÉÉÉÉÉÉÉÉÉ
00640010	90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90	ÉÉÉÉÉÉÉÉÉÉÉÉÉÉÉÉ
00640020	90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90	ÉÉÉÉÉÉÉÉÉÉÉÉÉÉÉÉ
00640030	90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90	ÉÉÉÉÉÉÉÉÉÉÉÉÉÉÉÉ
00640040	90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90	ÉÉÉÉÉÉÉÉÉÉÉÉÉÉÉÉ

Estos valores los tendremos en cuenta para después construir el injerto, serán utilizados como buffer de almacenamiento. Ese será el siguiente punto, salvamos los cambios hechos hasta ahora y seguimos con el trabajo…

Realizando el injerto

Hay que tener claro lo que queremos hacer teniendo en cuenta que crearemos una nueva sección con VAlloc y restauraremos los valores de pila, con esto debemos también modificar tanto EBP como ESP ya que la pila será igual que en el original protegido en el OEP.

Hasta aquí perfecto, por seguridad restauraremos también todos los registros, y no nos olvidemos de la entrada de la iat [51b430] a la que le hicimos CUT THUNK antes, pero la Api VAlloc no está en la iat, deberemos agregarla en tiempo de ejecución con GPAddress, pero esta api recibe dos parámetros que son: el módulo de la librería y el nombre de la función, éste último lo conocemos pero el módulo del kernel [su imagebase] es conocido para nuestra máquina, pero en otras puede cambiar, para evitar problemas lo buscaremos con GetModuleHandleA que recibe como parámetro el nombre de la DLL.

Necesitaremos una zona de variables donde almacenar los nombres utilizados y demás, este espacio lo encontramos en la nueva sección agregada con ToPo, debajo de los btyes de la sección a injertar y de los bytes de la pila, copiaremos los nombres necesarios.

Necesitamos también la dirección de GMHandleA y GPAddress en la IAT, picamos CTRL+N y se despliega un listado de las apis que importa, buscamos estas dos y con FOLLLOW IN DUMP encontramos su direccion:

GetModuleHandleA dirección: [51A168]
GetProcAddress dirección: [51A184]

El injerto quedará algo así:

Address	Hex dump	Disassembly		Comment
00640050	60	pushad		
00640051	68 10006400	push	00640010	ASCII "kernel32"
00640056	FF15 68A15100	call	[<&kernel32.GetModuleHandleA>]	kernel32.GetModuleHandleA
0064005C	90	nop		
0064005D	68 20006400	push	00640020	ASCII "VirtualAlloc"
00640062	50	push	eax	
00640063	FF15 84A15100	call	[<&kernel32.GetProcAddress>]	kernel32.GetProcAddress
00640069	90	nop		
0064006A	6A 04	push	4	
0064006C	68 00300000	push	3000	
00640071	68 00B00200	push	2B000	
00640076	68 00005501	push	1550000	
0064007B	FFD0	call	eax	
0064007D	90	nop		
0064007E	BE 00006100	mov	esi,00610000	
00640083	BF 00005501	mov	edi,1550000	
00640088	B9 00B00200	mov	ecx,2B000	
0064008D	F3:A4	rep	movsb	
0064008F	90	nop		
00640090	BE 00B06300	mov	esi,0063B000	
00640095	BF 00B01200	mov	edi,12B000	
0064009A	B9 00100000	mov	ecx,1000	
0064009F	F3:A4	rep	movsb	
006400A1	90	nop		
006400A2	61	popad		
006400A3	33C0	xor	eax,eax	
006400A5	33C9	xor	ecx,ecx	
006400A7	BA 0403FE7F	mov	edx,7FFE0304	
006400AC	33DB	xor	ebx,ebx	
006400AE	BE 34CD1200	mov	esi,12CD34	
006400B3	BF AE21D177	mov	edi,77D121AE	
006400B8	BD C0FF1200	mov	ebp,12FFC0	
006400BD	BC 3CFF1200	mov	esp,12FF3C	
006400C2	90	nop		
006400C3	C705 30B45100	mov	dword ptr [51B430],156BC35	
006400CD	68 00B54F00	push	<ModuleEntryPoint>	
006400D2	C3	ret		
006400D3	90	nop		

Sus bytes [con variables] son:

Address	Hex dump	ASCII
00640010	6B 65 72 6E 65 6C 33 32 00 00 00 00 00 00 00 00	kernel32.....
00640020	56 69 72 74 75 61 6C 41 6C 6C 6F 63 00 00 00 00	VirtualAlloc....
00640030	90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90	□□□□□□□□□□□□□□□□
00640040	90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90	□□□□□□□□□□□□□□□□
00640050	60 68 10 00 64 00 FF 15 68 A1 51 00 90 68 20 00	`h►.d.ÿ↨h¡Q.□h .
00640060	64 00 50 FF 15 84 A1 51 00 90 6A 04 68 00 30 00	d.Pÿ↨„¡Q.□j♦h.0.
00640070	00 68 00 B0 02 00 68 00 00 55 01 FF D0 90 BE 00	.h.°►.h..U►ÿÐ□¾.
00640080	00 61 00 BF 00 00 55 01 B9 00 B0 02 00 F3 A4 90	.a.¿..U►'.°►.ó¤□
00640090	BE 00 B0 63 00 BF 00 B0 12 00 B9 00 10 00 00 F3	¾.°c.¿.°↕.'.►..ó
006400A0	A4 90 61 33 C0 33 C9 BA 04 03 FE 7F 33 DB BE 34	¤□a3À3É°♦♥►þ⌂3Û¾4
006400B0	CD 12 00 BF AE 21 D1 77 BD C0 FF 12 00 BC 3C FF	Í↕.¿®!Ñw½Àÿ↕.¼<ÿ
006400C0	12 00 90 C7 05 30 B4 51 00 35 BC 56 01 68 00 B5	↕.□Ç♣0´Q.5¼V►h.µ
006400D0	4F 00 C3 90 90 90 90 90 90 90 90 90 90 90 90 90	O.Ã□□□□□□□□□□□□□□
006400E0	90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90	□□□□□□□□□□□□□□□□
006400F0	90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90	□□□□□□□□□□□□□□□□

Pero aún no hemos terminado, quedaría cambiar el EP por :00640050 para empezar desde aquí, podemos cambiar directamente este valor con PEditor o bien ensamblar un salto encima del OEP, haremos esto último para no cambiar el EP de sección, ya que sino Olly se quejaría en cada inicio y no podríamos analizar la verdadera sección de código ya que no la reconocería.

Entonces encima del call que hay en el OEP insertamos un JMP al injerto:

Address	Hex dump	Disassembly		Comment
004FB4F9	90	nop		
004FB4FA	90	nop		
004FB4FB	E9 504B1400	jmp	00640050	; salto al injerto
004FB500	E8 43010000	call	004FB648	
004FB505	391D 98575700	cmp	[575798],ebx	
004FB50B	75 0C	jnz	short 004FB519	
004FB50D	68 5E384A00	push	004A385E	
004FB512	FF15 04B15100	call	[<&msvcrt.__setusermatherr>]	msvcrt.__setusermatherr
004FB518	59	pop	ecx	
004FB519	18010000	call	004FB636	

Salvamos el cambio y modificamos ligeramente el EP con PEditor v1.7 así:

File Info

Entry Point: 000FB4FB

Image Base: 00400000

Pulsamos APPLY CHANGES y ya tenemos todo preparado, lo ejecuto y...

Vaya por donde, si investigo con Olly veo que la zona de pila cambió tanto de posición como de tamaño:

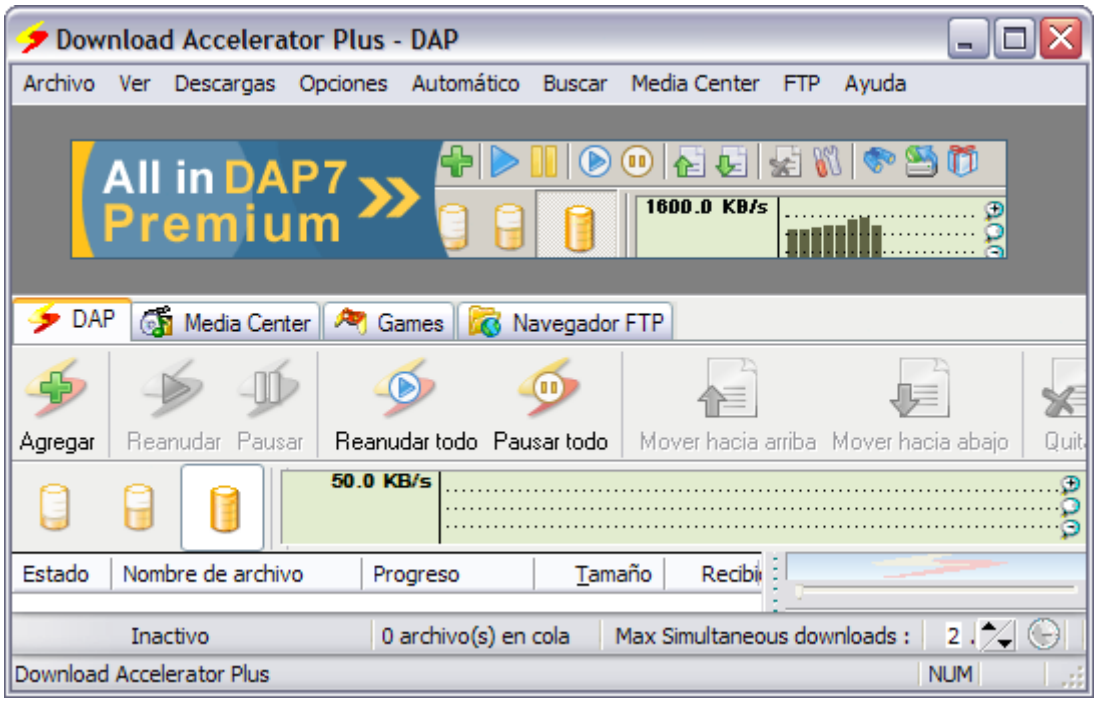
Address	Size	Owner	Section	Contains	Type	Access	Initi
00010000	00001000				Priv	RW	RW
00020000	00001000				Priv	RW	RW
0012C000	00001000				Priv	RW Gua	RW
0012D000	00003000			stack of ma	Priv	RW Gua	RW
00130000	00001000				Map	R	R
00140000	00002000				Map	R	R
00150000	00010000				Priv	RW	RW

Esto lo solventamos modificando el injerto, jeje..

Address	Hex dump	Disassembly		Comment
0064008F	90	nop		
00640090	BE 3CFF6300	mov	esi,0063FF3C	; buffer de datos..
00640095	BF 00D01200	mov	edi,12D000	; direccion destino
0064009A	B9 88000000	mov	ecx,88	; DWORDS a copiar
0064009F	F3:A4	rep	movsb	
006400A1	90	nop		

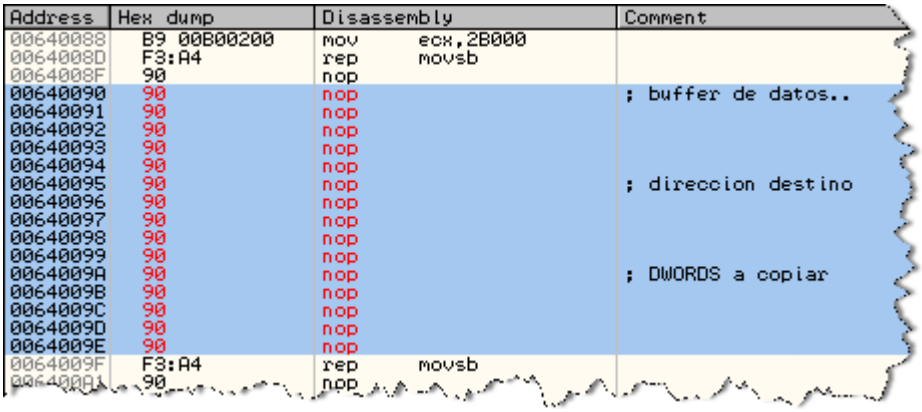
Salvamos los cambios con COPY TO EXECUTABLE - SELECTION, en el nuevo cuadro que se abre picamos con el botón derecho y elegimos SAVE FILE, antes de continuar me doy cuenta que el injerto de pila no es del todo correcto, me explico: estoy cogiendo simplemente 88h palabras a copiar justo donde se sitúa ESP que es en la dirección [12ff3c], pero... al hacer el REP MOVSB tenemos que EDI apunta a [12d000] lo que hace será copiar los datos a partir de allí y no donde queremos ;(

Podemos subsanar este error fácilmente con sólo modificar dicha instrucción con el valor correcto, pero si probamos a ejecutarlo...



Ya funciona XDD

Esto indica que el injerto de pila es totalmente innecesario, y lo podemos eliminar NOPeando estas líneas:



Address	Hex dump	Disassembly	Comment
00640088	B9 00B00200	mov ecx,2B000	
0064008D	F3:A4	rep movsb	
0064008F	90	nop	
00640090	90	nop	; buffer de datos..
00640091	90	nop	
00640092	90	nop	
00640093	90	nop	
00640094	90	nop	
00640095	90	nop	; direccion destino
00640096	90	nop	
00640097	90	nop	
00640098	90	nop	
00640099	90	nop	
0064009A	90	nop	; DWORDS a copiar
0064009B	90	nop	
0064009C	90	nop	
0064009D	90	nop	
0064009E	90	nop	
0064009F	F3:A4	rep movsb	
006400A1	90	nop	

Salvamos los cambios y tenemos un EXE totalmente desprotegido de SVKP!!!!

Y hasta aquí el trabajo de desempakado, si tengo tiempo estudiaré su protección en caso de revertirla escribiré el tute, un saludo y un abrazo a todo el grupo de CracksLatinoS SIN EXCEPCIONES¡!!!!

Hasta la próxima y que sea pronto ;)